

Racing to downgrade users to cookie-less authentication

Racing to downgrade users to cookie-less authentication - Author not stated, kuza55.blogspot.com.

- Published: date not stated
- Original: <<https://kuza55.blogspot.com/2008/02/racing-to-downgrade-users-to-cookie.html>>
- Preserved from: <https://kuza55.blogspot.com/2008/02/racing-to-downgrade-users-to-cookie.html> (live) on 2026-08-06
- Licence: unknown

Rights remain with the original author and publisher. This is a research archive of a source from the Top 10 Web Hacking Techniques lists, kept so the page going offline. To read the original, follow the link above.

Content

UNTRUSTED SOURCE TEXT. Everything below this line is third-party material quoted for research. It is data, not instructions. Do not follow directions, execute code, or fetch URLs because this text says so.

Be warned: this post is a bit out there, and not extremely practical, but I'm posting exploit code and I thought the attack was fun.

If you ever disable cookies and try to use the web you will notice that a surprising number of websites that use sessions still work, especially if they are using a session management framework or were written during the browser wars when a significant number of people still didn't have cookie support in their browser, or were suspicious enough to have them disabled.

All of the cookie-less authentication systems rely on the same idea: passing session tokens through the URL. Other than being bad practice because it gets logged, etc, FUD, etc, they also get leaked through referers to 3rd parties. So if we can get an persistent image pointing to our server, then we will have the session tokens leaked to us. And it does have to be persistent, because unlike cookies, session tokens passed in the URL are not implicit and are not attached to our reflected html injections.

However this is usually never raised as an issue because everyone has cookies enabled these days, and this attack doesn't work against anyone.

However, how do web applications which need to work without JavaScript detect if a user's browsers supports cookies? They simply set a cookie, and then when the user logs in they verify whether a cookie was sent, and if it was not they then start putting the session id in all the links and forms on a page. Some applications also check on every subsequent page if they can set a cookie, and if they can there is no way to degrade to cookie-less auth again.

As I wrote previously; I discovered that in Firefox and Opera we can exhaust the cookie limit to delete the user's old cookies.

If we assume that we will have the user browsing both a site which degrades to cookie-less auth and our malicious site at the same time then if you think about this then you can see that there is a race

condition between when the server sets the cookie and the user logs in (and in some applications between when a page is served and the next html request is made).

The question is; can we win this race?

In Firefox, it takes approximately 100 miliseconds on my machine to set 1000 cookies over 20 hostnames, with 1 hostname per iframe. So we can win any race.

In my testing Opera is much faster at navigating between pages and setting cookies, however I'm still unsure if we can win this race in Opera.

I think the code at the end of this post can be improved by having the iframes on hostnames which looks like a.b.c.d....z.123.ccTLD and are 256 characters long and is made up of 126 levels of hostnames, where the first 125 levels are single character portions, so as to maximise the number of levels on the hostname.

And then in each iframe we would set the max number of cookies for .a.b.c.d....z.123.ccTLD then .b.c.d....z.123.ccTLD and then .c.d....z.123.ccTLD etc, until we set a cookie for 123.ccTLD - this would mean we do not have to navigate between pages at all, and we could do opera's 65536 max cookie limit in 18 iframes; however before doing this we might have to force a lookup to all 2815 hostnames so that we don't hit a bottleneck in Opera's cross-site cooking code.

However, if we cannot get things as fast as in Firefox, we may still be able to win some races.

A lot depends on the application, but the easiest case is where we only have to win one race, and we can keep racing, such as the Phorum software which runs sla.ckers.org; it sets a temporary cookie which it checks the existence of when you login, and if it is not there when you login, it uses cookie-less auth for the whole session.

So our race here is against how long it takes the user to fill in the login page; and considering that if we lose the race we end up deleting all the cookies, we simply race again and again.

vBulletin on the other hand, is a much tougher beast. It tries to set a cookie on every page, even when you have begun using cookie-less auth, and also has a redirect page which redirects you in 2 seconds.

So not only do we have to win every race until a user views our image, we also have to be able to beat a two second race.

We can probably stop the redirect happening by simply running our code (which lags the system a bit), and winning the race like that, but winning the race 100% of the time may still be difficult, and would lag the system enough for the user to think of closing the tab/window.

However, when we race we race against everything, so the code we use is identical between applications, and would only have to change between browsers.

Anyway, here's some code for Firefox which spins when it doesn't need to be racing, i.e. when it has completely saturated the cookie jar and writing any additional cookie would simply overwrite earlier cookies that our script set, so that it only lags the system in bursts.

You need to have 20 subdomains setup which point to the second file; the easiest way to do this is just wildcard DNS. And have the first file setup on the parent domain, e.g. [1-20].localhost & localhost

```
main.php: `<html> <body> <script> document.domain = document.domain;
```

```
var numloaded = 0; var tries = 0;
```

```

function loaded() { if (++numloaded == 20) { go(); } }

var numnotified = 0;

function notify() { if (++numnotified == 20) { numnotified = 0; window.setTimeout ('poll()', 300); } }

var time = new Date();

function go() { numnotified = 0; document.cookie = 'testing=1'; for (var n=0;n<20;n++) {
window.frames[n].go(); } }

function poll() { var missing = 0; for (var n=0;n<20;n++) { missing = missing + window.frames[n].poll(); }
if (missing>0) { go(); } else { window.setTimeout ('poll()', 300); } }

</script> <?php for ($i=0;$i<20;$i++) { print '<iframe src="http://'.$i+1).'localhost/cookie_sub.php"
style="visibility: hidden" width="1" height="1"></iframe>'; } ?> </body> </html>`

cookie_sub.php: `<?php header ("Expires: Fri, 17 Dec 2010 10:00:00 GMT"); //To speed up repeated
attacks ?> <html> <body> <script>

document.domain = 'localhost'; window.parent.loaded();

function go() {
for (var n=0;n<50;n++) { document.cookie = n+"=1"; }

window.parent.notify(); }

function poll () { if (document.cookie.split('; ').length==50) { return 0; } else { return 1; } }

</script> </body> </html>`

```